

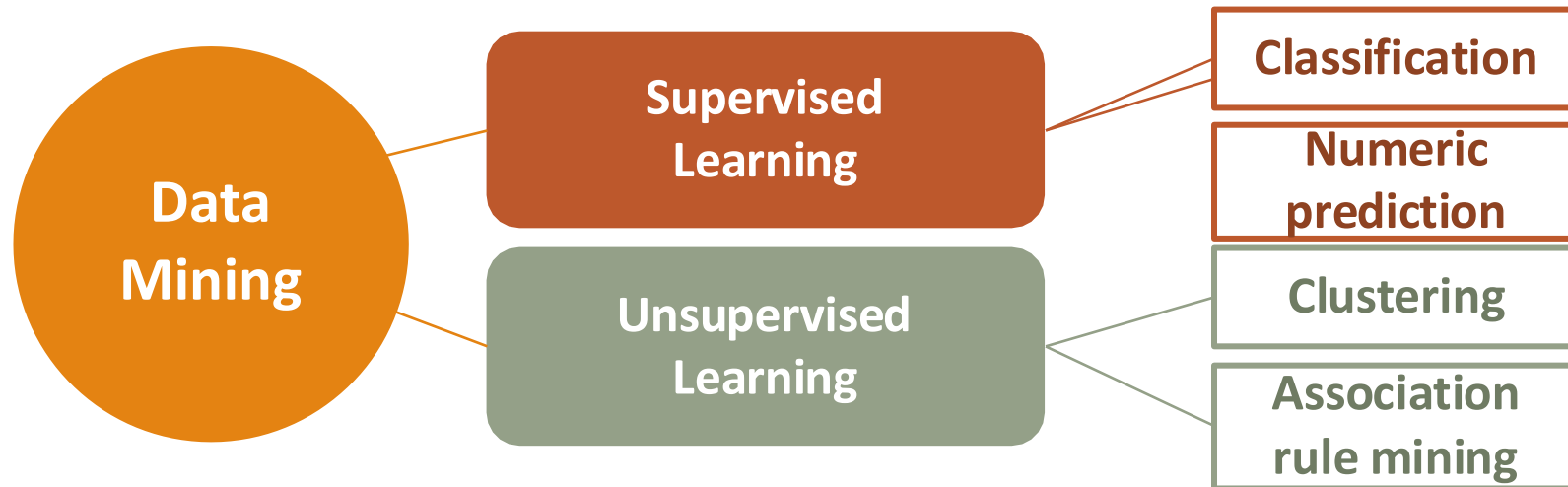
Lecture 2: Managing and Understanding Data

Concepts: Recap

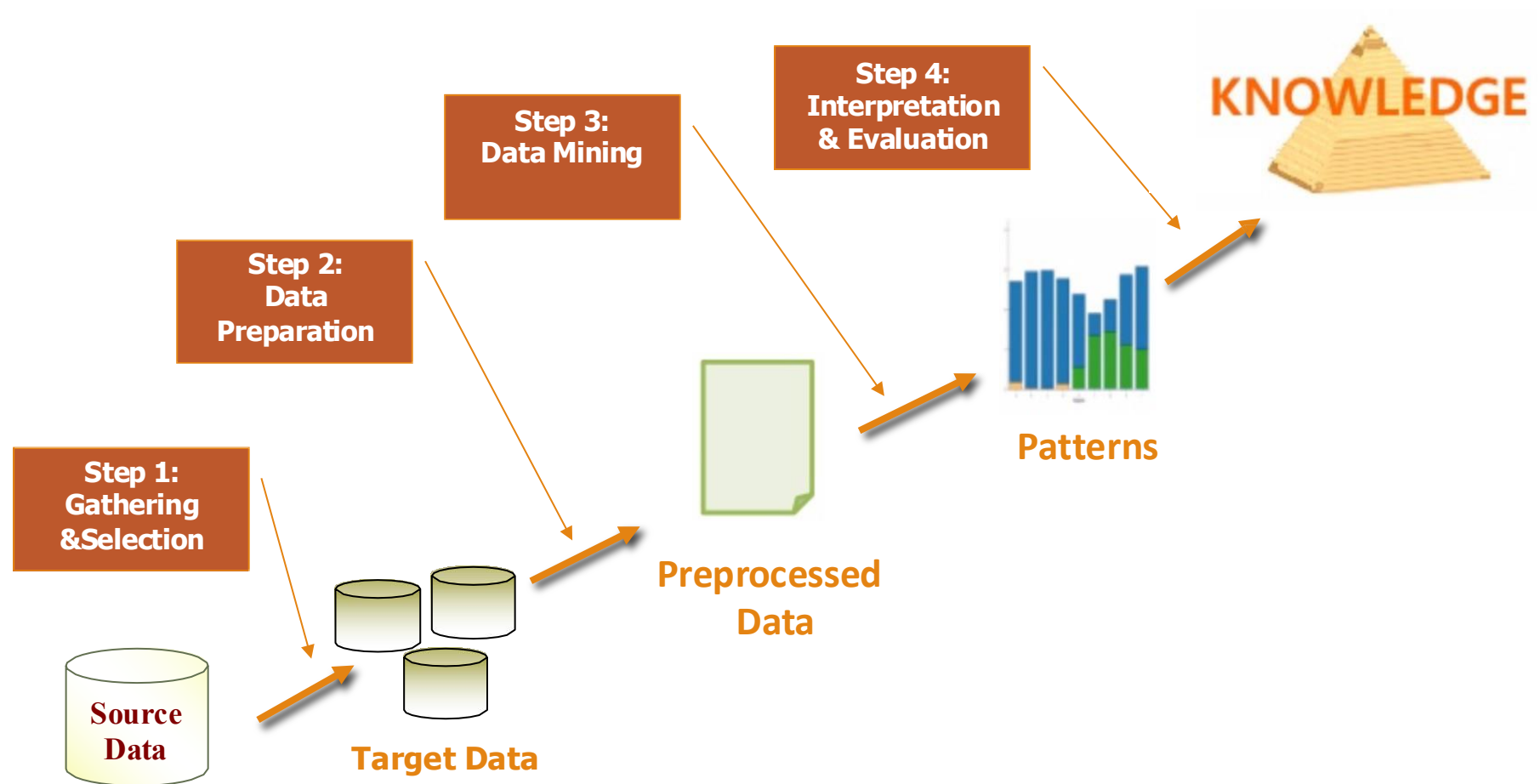
■ Concepts in data mining:

- Data
- Instance/record/example/data point/observation/row
- Variable/feature/attribute/column
- Variable type
 - Categorical variable
 - **Nominal** variables are variables that have two or more categories, but which do not have an intrinsic order.
 - **Ordinal** variables are variables that have two or more categories just like nominal variables only the categories can also be ordered or ranked.
 - Numeric variable

Data Mining Tasks: Recap

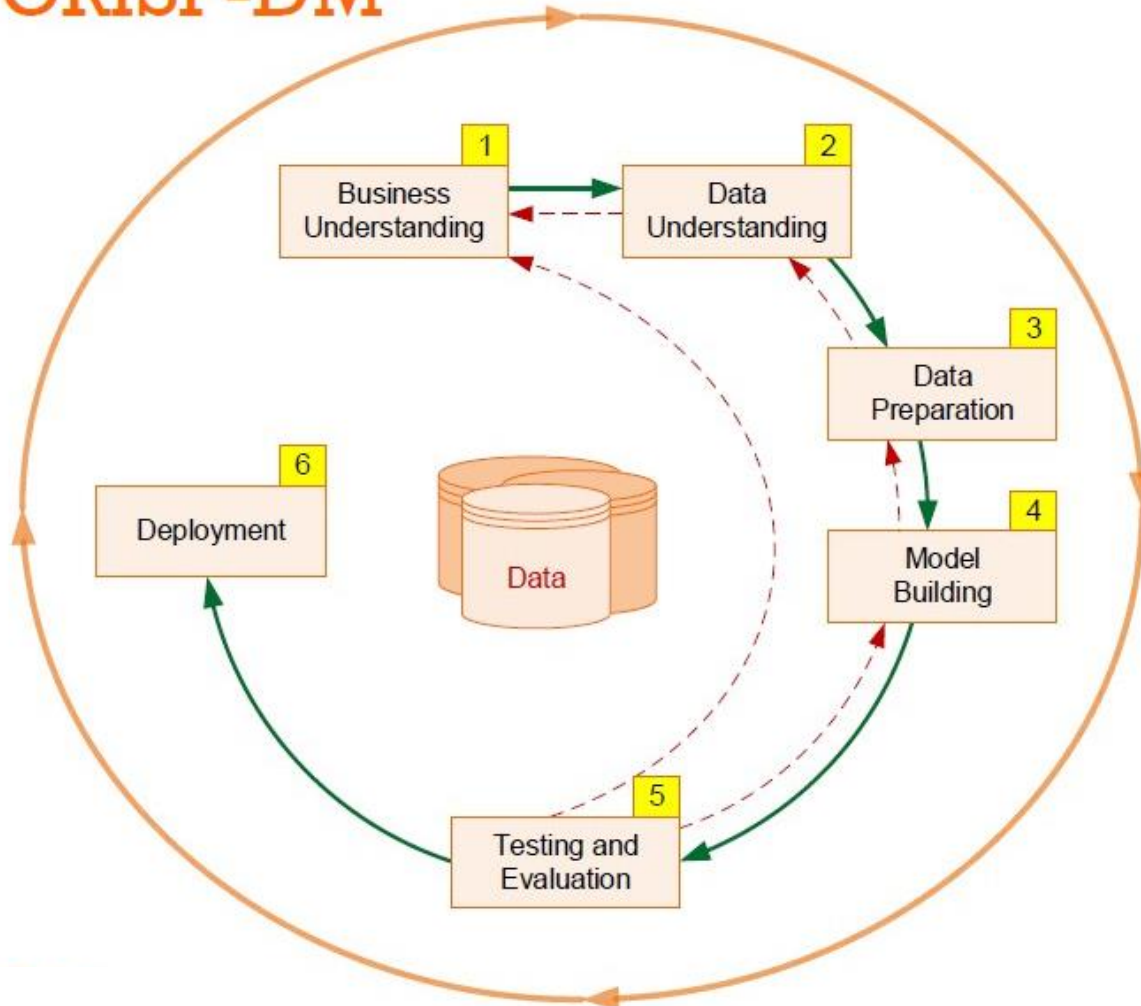


Business Analytics: Recap



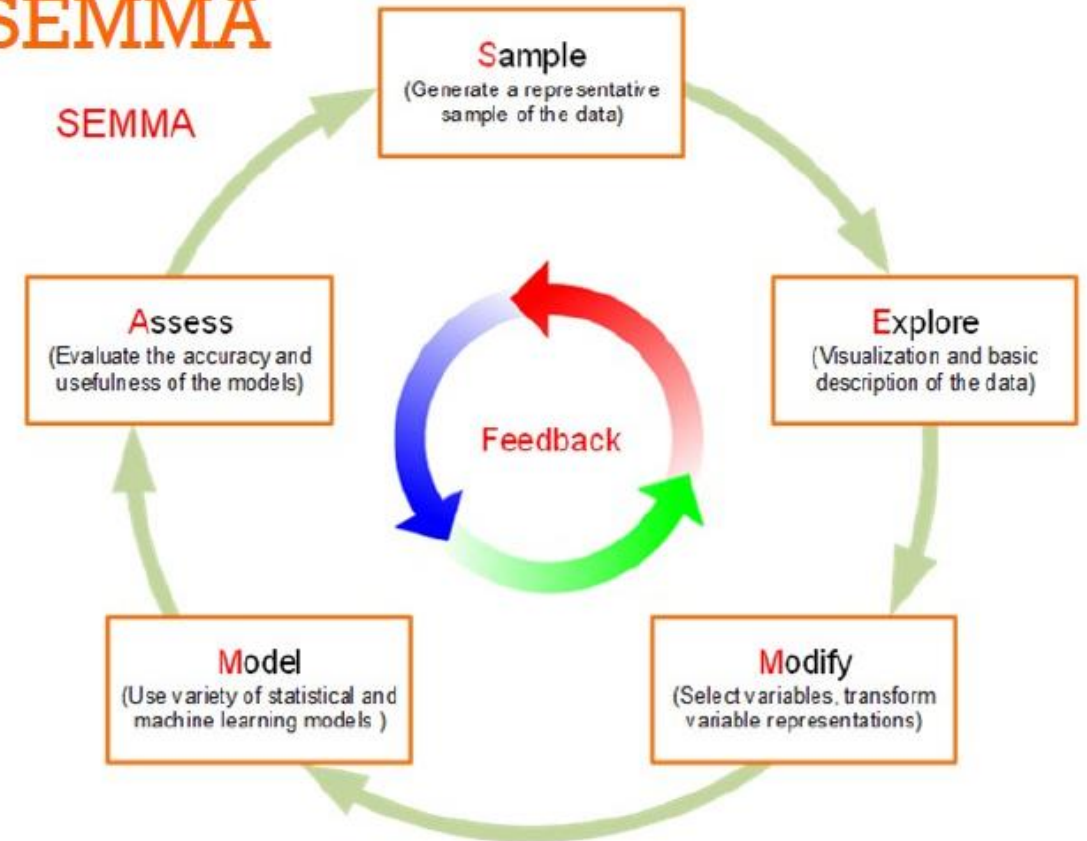
Business Data Analytics Process

CRISP-DM



SEMMA

SEMMA



Business Understanding

- The Cornerstone of Successful Analytics
- Clarifying the **business objectives**
- Assessing the **current situation**
- Defining **data mining goals** aligned with business goals
- Establishing **success criteria**
- Producing an initial **project plan**

Business Understanding

- **Be objective and open minded**
- Every project must start from an open mind and a clean slate
- Dig deep to find the real problem (not the symptoms)
- **Study documented knowledge/information**
- Definitions, policies, procedures, etc.
- **Ask subject matter experts (SMEs)**
- But do not believe in everything you hear!
- **Establish the objective**
- Build consensus on business problem and objective(s)

Data Understanding

- Knowing and validating the definitions, meaning, and measurement of each variable in the dataset.
- Understanding **what each variable represents** in the real world
- Knowing **how it is measured or generated**
- Identifying the **true data type** (not just how it is stored)
- Clarifying units, scales, coding, and allowable values
- Detecting ambiguous or overloaded variables (same name, different meaning)

Data Understanding

- Prevents **misinterpretation** of features
- Avoids modeling errors (e.g., treating IDs or codes as numeric measures)
- Ensures correct handling of **missing values, outliers, and transformations**

Data Understanding

- **Identify all the data sources**
- Within the organization and outside the organization
- **Become familiar with the terminology (domain lingo)**
- Determine the genuine types of the data features
- **Don't be distracted by the domain experts' biases on**
- What is important and what is not
- What is accessible/allowed and what is not
- **Keep in mind that data is the key to success**
- Large, feature-rich, high-quality data is the most critical input

Overview: Some Basic Steps To Understand Data

- Exploring the Structure of Data
- Exploring Numeric Variables
- Exploring Categorical Variables
- Exploring Relationships between Variables

Overview

- Exploring the Structure of Data
- Exploring Numeric Variables
- Exploring Categorical Variables
- Exploring Relationships between Variables

Exploring the Structure of Data

We can display a data frame with Python code

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False
3	4	Mega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False
4	5	Charmander	Fire	NaN	39	52	43	60	50	65	1	False
...	...	...	...	...	...	...	...	...	...	...	...	...
795	796	Diancie	Rock	Fairy	50	100	150	100	150	50	6	True
796	797	Mega Diancie	Rock	Fairy	50	160	110	160	110	110	6	True
797	798	Hoopa Confined	Psychic	Ghost	80	110	60	150	130	70	6	True
798	799	Hoopa Unbound	Psychic	Dark	80	160	60	170	130	80	6	True
799	800	Volcanion	Fire	Water	80	110	120	130	90	70	6	True

Number of instances

Number of variables

The type of each variable

800 rows × 12 columns

Features of One Pokemon (e.g., Pikachu)

Name: Pikachu

Attack: 55

Defense: 40

Sp. Atk: 50

Sp. Def: 50

Speed: 90

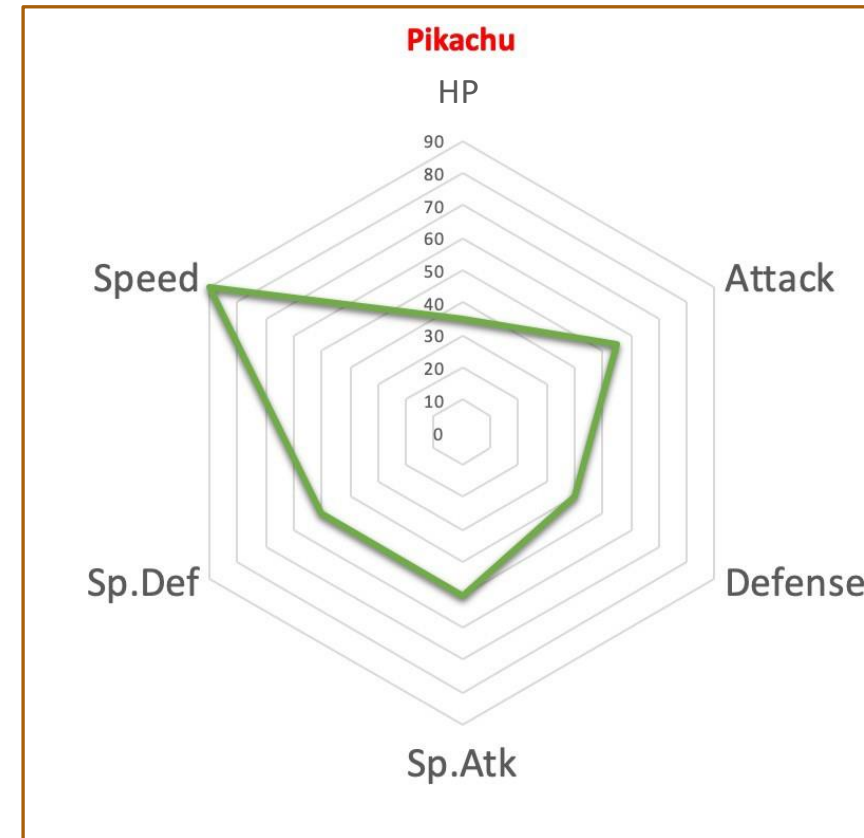
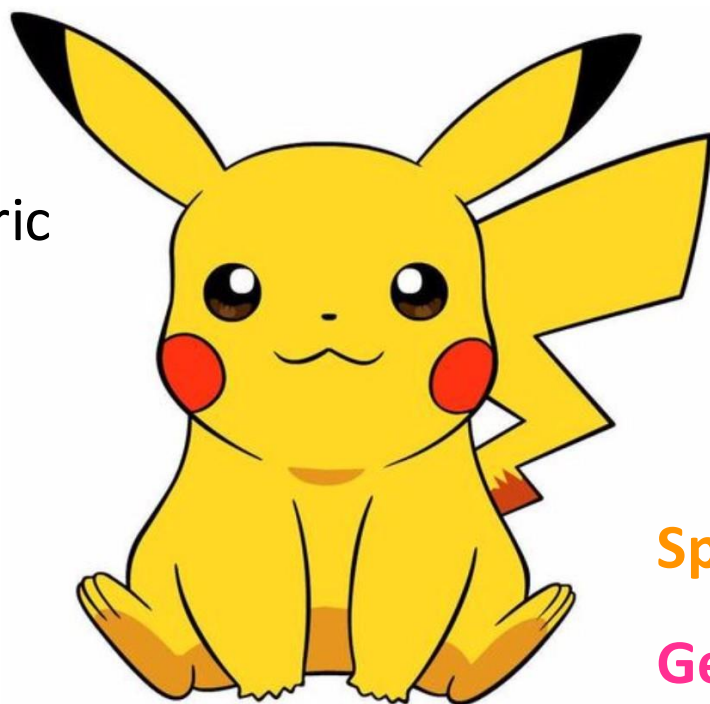
Generation: 1

Legendary: False

Type 1: Electric

Type 2: None

HP: 35



Exploring the Structure of Data

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False
3	4	Mega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False
4	5	Charmander	Fire	NaN	39	52	43	60	50	65	1	False
...	...	...	...	...	...	...	...	...	...	...	...	...
795	796	Diancie	Rock	Fairy	50	100	150	100	150	50	6	True
796	797	Mega Diancie	Rock	Fairy	50	160	110	160	110	110	6	True
797	798	Hoopa Confined	Psychic	Ghost	80	110	60	150	130	70	6	True
798	799	Hoopa Unbound	Psychic	Dark	80	160	60	170	130	80	6	True
799	800	Volcanion	Fire	Water	80	110	120	130	90	70	6	True

800 rows × 12 columns

Exploring the Structure of Data

- Remove variable (dimensionality reduction)
- Change/fix categorical variables to "category."

```
# Display all numeric variables  
pokemon.select_dtypes(include=[ 'number' ])
```

	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
0	45	49	49	65	65	45
1	60	62	63	80	80	60
2	80	82	83	100	100	80
3	80	100	123	122	120	80
4	39	52	43	60	50	65
...	...	...	...	...	...	...
795	50	100	150	100	150	50
796	50	160	110	160	110	110
797	80	110	60	150	130	70
798	80	160	60	170	130	80
799	80	110	120	130	90	70

800 rows x 6 columns

```
# Display all categorical variables  
pokemon.select_dtypes(include=[ 'category' ])
```

	Type 1	Type 2	Generation	Legendary
0	Grass	Poison	1	False
1	Grass	Poison	1	False
2	Grass	Poison	1	False
3	Grass	Poison	1	False
4	Fire	NaN	1	False
...	...	...	...	...
795	Rock	Fairy	6	True
796	Rock	Fairy	6	True
797	Psychic	Ghost	6	True
798	Psychic	Dark	6	True
799	Fire	Water	6	True

800 rows x 4 columns

Dimensionality Reduction

- **Dimensions = features / variables**
 - High-dimensional data → many features (e.g., hundreds of clinical variables per patient)
 - Reduces **noise** and **redundancy**
 - Improves **model performance**
 - Speeds up **training and computation**
 - Helps **visualization** and **interpretability**
 - Mitigates the **curse of dimensionality**
- Correlation filtering
 - Statistical tests
 - LASSO $\text{Loss} = \text{Error} + \lambda \sum |\beta_j|$
 - Tree-based importance

Other reasons to remove a feature

- **Data Quality Issues**

A lab test with 70% missing or inconsistent values

- **Data Leakage**

Length of hospital stay used to predict mortality at admission

- **Low Variance**

A column where 99.9% of values are “Yes.”

- **Redundancy / Duplicates**

Height in inches and height in centimeters

- **Interpretability & Simplicity**

A complex engineered feature that clinicians cannot explain

- **Ethical / Legal / Privacy Concerns**

Social Security Number or exact home address

- **Operational Constraints**

A biomarker only available days after prediction is needed

- **Model-Specific Issues**

A variable causing multicollinearity in model

- **High Cardinality (Categorical Features)**

User ID with millions of unique values

- **Lack of Business Relevance**

Internal system log codes unrelated to decision-making

Feature Extraction and Engineering

Models learn from features — feature engineering teaches them what matters.

- **Feature Engineering:**

Creating **new features** from raw data to improve model performance (Uses **domain knowledge** + data transformations)

- **Feature Extraction:**

Transforming original features into a **new feature space** (reduces noise and captures hidden structure)

Feature Extraction and Engineering

- **Aggregation**

Average lab values over the first 24 hours

- **Transformation**

Log-transform skewed medical costs

- **Interaction Features**

Age × BMI to capture combined risk

- **Encoding Categorical Variables**

One-hot encoding blood type

- **Temporal Features**

Time since last hospital admission

- **Statistical Features**

Mean, variance, and trend of vital signs

- **Domain-Specific Features**

Comorbidity index derived from diagnoses

- **Dimensionality Reduction
(Extraction)**

PCA combining correlated lab tests

Exploring the Structure of Data

- Missing values
 - Missing mechanism: understanding the reasons why data are missing
 - Deal with missingness

	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False
1	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False
2	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False
3	Mega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False
4	Charmander	Fire	None	39	52	43	60	50	65	1	False
...	...	...	...	...	...	...	...	...	...	...	...
795	Diancie	Rock	Fairy	50	100	150	100	150	50	6	True
796	Mega Diancie	Rock	Fairy	50	160	110	160	110	110	6	True
797	Hoopa Confined	Psychic	Ghost	80	110	60	150	130	70	6	True
798	Hoopa Unbound	Psychic	Dark	80	160	60	170	130	80	6	True
799	Volcanion	Fire	Water	80	110	120	130	90	70	6	True

800 rows x 11 columns

Exploring the Structure of Data

- Missing values
 - Missing mechanism: understanding the reasons why data are missing
 - Deal with missingness

```
# Examine missing values
```

```
pokemon.isnull().sum()
```

```
#          0
Name      1
Type 1    0
Type 2    386
HP         0
Attack    0
Defense   0
Sp. Atk   0
Sp. Def   0
Speed     0
Generation 0
Legendary 0
dtype: int64
```

- Is this enough to find the missing?!
- There are usually different missing patterns depending on the analysis

Common Ways to Handle Missingness

- Drop Rows**

Remove records with only a few missing values in a large dataset

- Drop Columns**

Remove a variable missing in 70% of observations

- Mean / Median Imputation**

Replace missing lab values with the median

- Mode Imputation (Categorical)**

Fill missing gender with most frequent category

- Constant / Flag Value**

Replace missing with “Unknown” or -1

- Model-Based Imputation**

Predict missing BMI using age and weight

- KNN Imputation**

Fill missing values using nearest patients

- Multiple Imputation**

Generate several plausible datasets and combine results

- Time-Series Imputation**

Forward-fill vital signs over time

- Missingness Indicators**

Add a binary flag: lab_test_missing = 1

Overview

- Exploring the Structure of Data
- **Exploring Numeric Variables**
- Exploring Categorical Variables
- Exploring Relationships between Variables

Exploring Numeric Variables

- We employ a common set of measurements to describe values known as **summary statistics**.

```
pokemon['Attack'].describe()
```

count	800.000000
mean	79.001250
std	32.457366
min	5.000000
25%	55.000000
50%	75.000000
75%	100.000000
max	190.000000

```
pokemon[['HP', 'Attack']].describe()
```

	HP	Attack
count	800.000000	800.000000
mean	69.258750	79.001250
std	25.534669	32.457366
min	1.000000	5.000000
25%	50.000000	55.000000
50%	65.000000	75.000000
75%	80.000000	100.000000
max	255.000000	190.000000

Exploring Numeric Variables

- Measures of **central tendency** are a class of statistics used to identify a value that falls in the middle of a set of data.
 - Mean: the sum of all values divided by the number of values
 - Median: the value that occurs halfway through an ordered list of values

$$\circ \text{median}(x) = \begin{cases} x_{(m+1)/2}, & \text{if } m \text{ is odd. } m = 2r + 1 \\ \frac{x_{(m/2)} + x_{(m/2+1)}}{2}, & \text{if } m \text{ is even. } m = 2r \end{cases}$$

Exploring Numeric Variables

- Mean and median are affected differently by the values falling at the far ends of the range.
 - Which one is more sensitive to **outliers**/extreme values?

```
pokemon[ 'Attack' ].describe()
```

count	800.000000	
mean	79.001250	← Mean
std	32.457366	
min	5.000000	
25%	55.000000	
50%	75.000000	← Median
75%	100.000000	
max	190.000000	

Exploring Numeric Variables

- The mean is highly sensitive to **outliers**, or values that are atypically high or low in relation to the majority of data. Because the mean is sensitive to outliers, it is more likely to be shifted higher or lower by a small number of extreme values.

Exploring Numeric Variables

■ Measuring **spread** – quartiles and the five-number summary

- Mean and median tell us little about whether or not there is diversity in the measurements.
- The **five-number summary** is a set of five statistics that roughly depict the spread of a feature's values.

1. Minimum (Min.)
2. First quartile, or Q1 (1st Qu.)
3. Median, or Q2 (Median)
4. Third quartile, or Q3 (3rd Qu.)
5. Maximum (Max.)

The **quartiles** divide a dataset into four portions, each with the same number of values.

Exploring Numeric Variables

```
pokemon[ 'Attack' ].describe()
```

count	800.000000
mean	79.001250
std	32.457366
min	5.000000
25%	55.000000
50%	75.000000
75%	100.000000
max	190.000000

- The difference between Q1 and Q3 is known as the **Interquartile Range (IQR)**

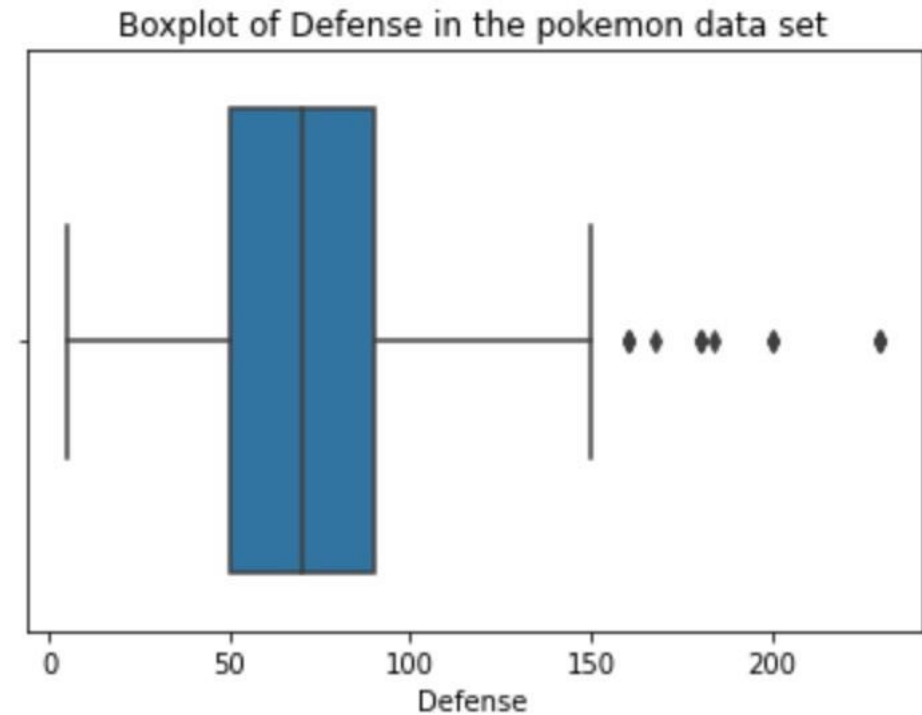
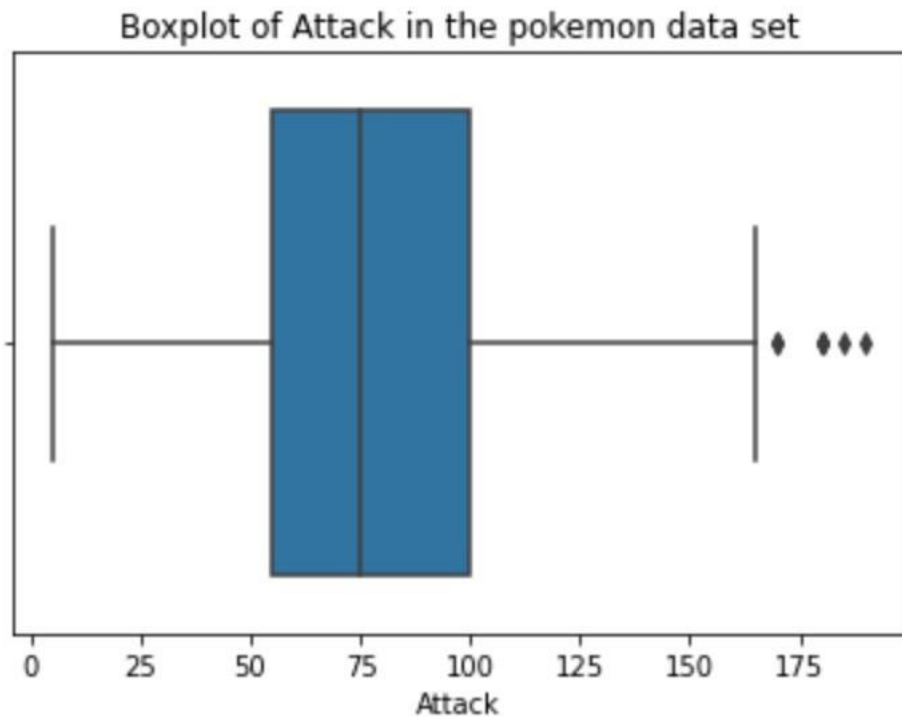
```
# IQR of Attack variable
```

```
IQR = pokemon[ 'Attack' ].quantile(0.75) - pokemon[ 'Attack' ].quantile(0.25)  
print("IQR:", IQR)
```

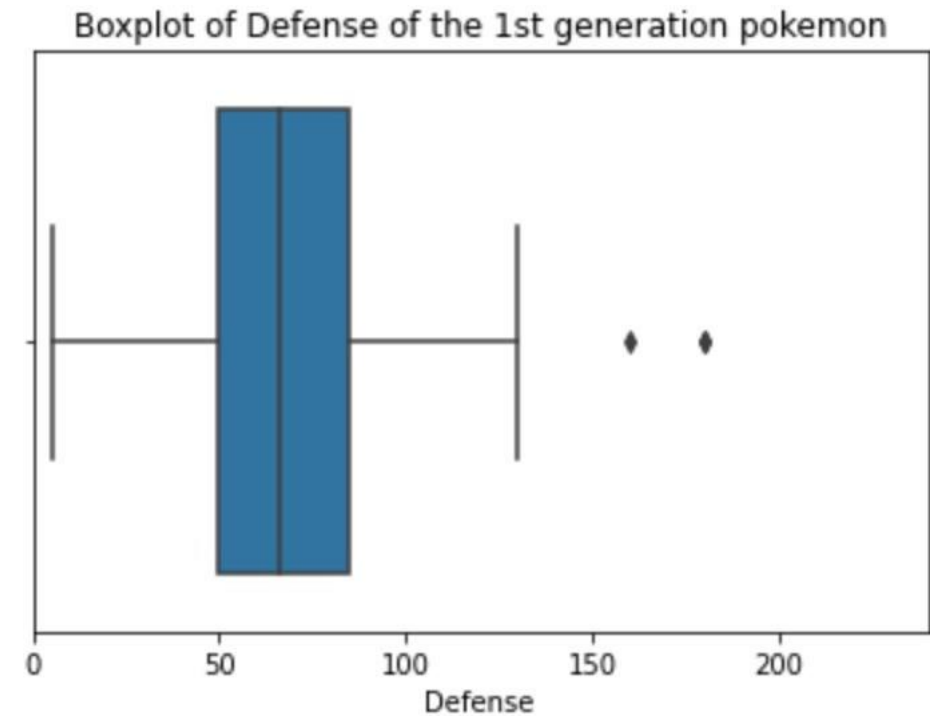
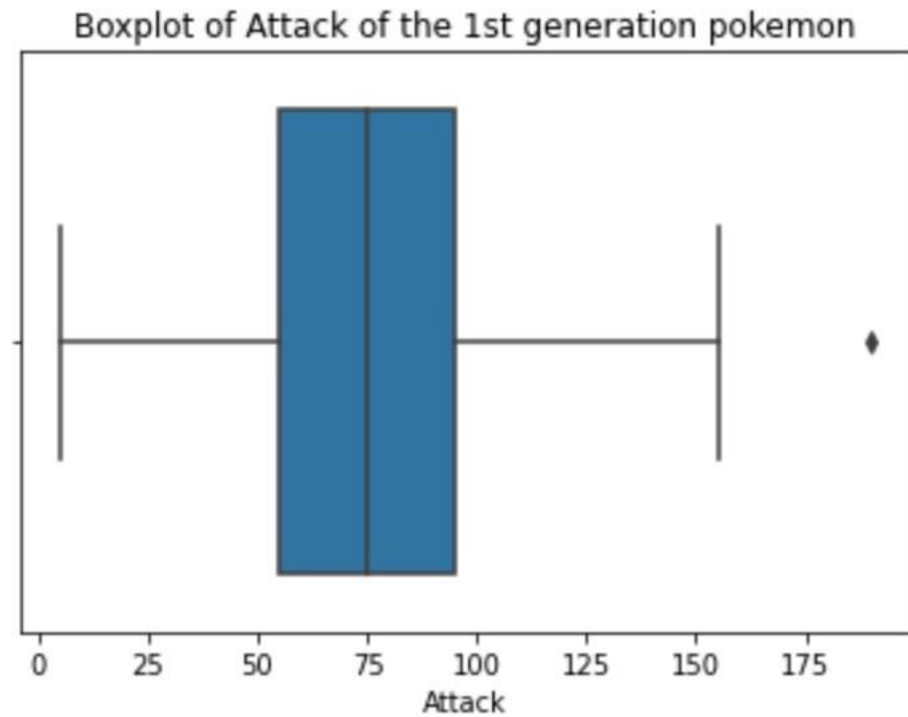
```
IQR: 45.0
```

Exploring Numeric Variables

- Visualizing numeric variables – boxplot
 - A common visualization of the five-number summary is **boxplot**.

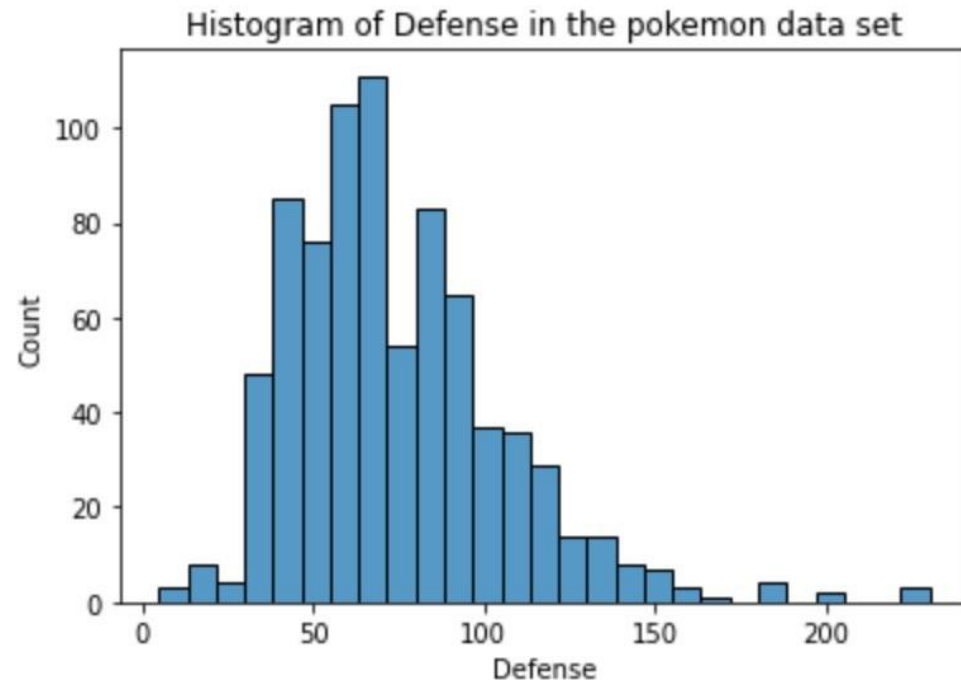
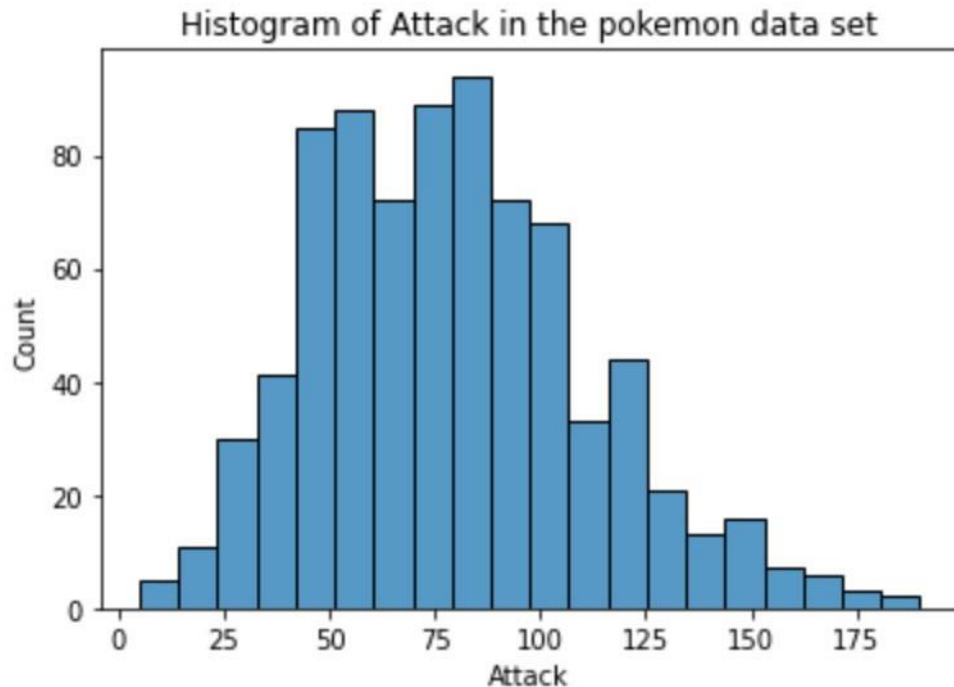


Exploring Numeric Variables



Exploring Numeric Variables

- Visualizing numeric variables – histograms
 - It divides the variable's values into a predefined number of portions or **bins** that act as containers for values.

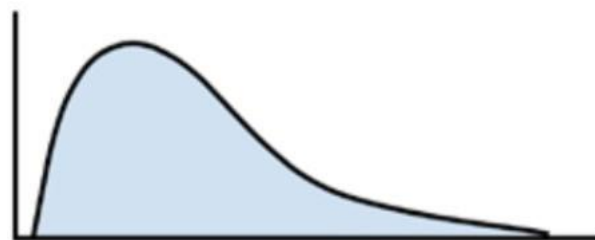


Exploring Numeric Variables

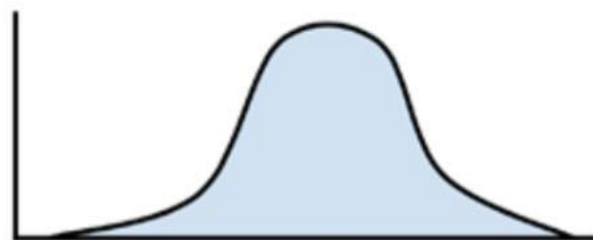
- Visualizing numeric variables – histograms
 - The histogram is composed of a series of bars with heights indicating the count, or **frequency** of values falling within each of the equal width bins partitioning the values.
 - The vertical lines that separate the bars, as labeled on the horizontal axis, indicate the start and end points of the range of values for the bin.

Exploring Numeric Variables

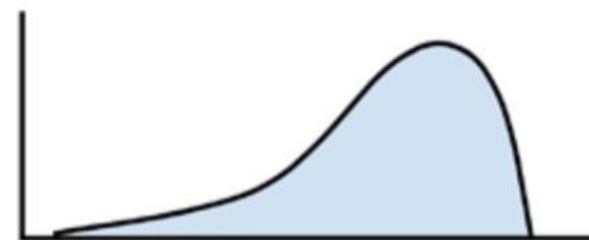
- This characteristic is known as **skew**, because the values on one end (left/right side) are far more spread out than the values on the other end (right/left side).



Right Skew



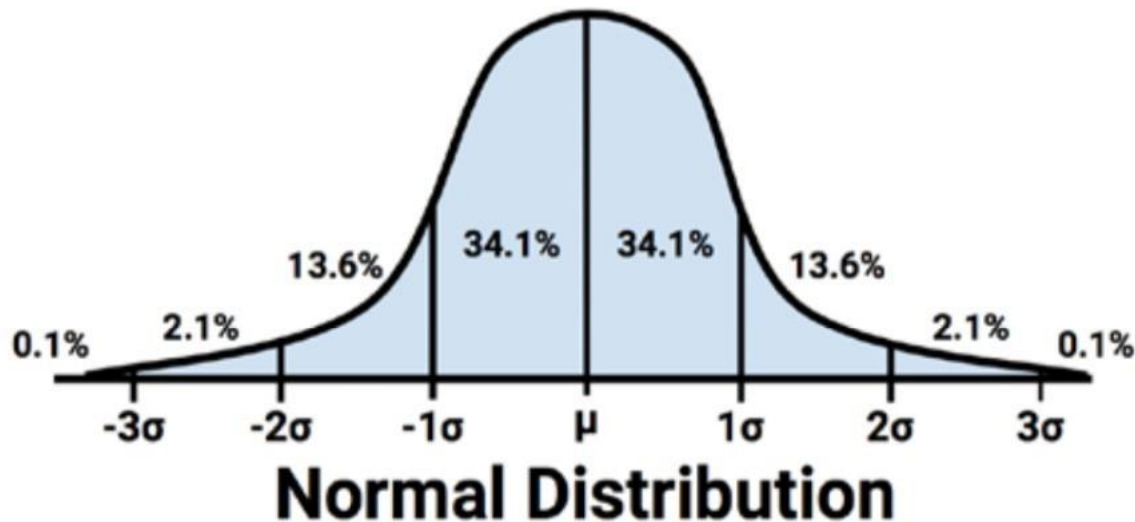
No Skew



Left Skew

Exploring Numeric Variables

- The **68-95-99.7 rule** states that 68 percent of the values in a normal distribution fall within one standard deviation of the mean, while 95 percent and 99.7 percent of the values fall within two and three standard deviations, respectively.



Overview

- Exploring the Structure of Data
- Exploring Numeric Variables
- Exploring Categorical Variables
- Exploring Relationships between Variables

Exploring Categorical Variables

■ Frequency of particular values

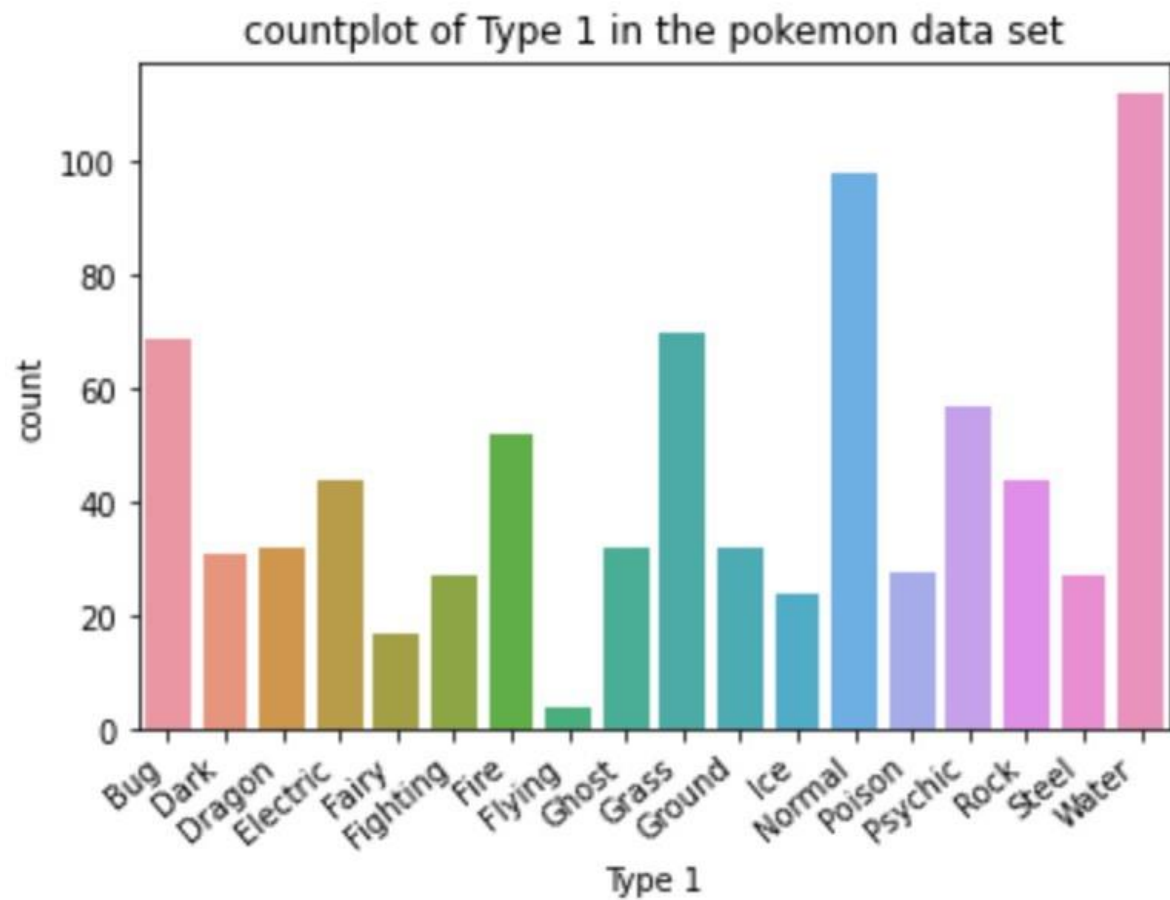
```
# Show the counts of unique pokemon types
pokemon['Type 1'].value_counts()
```

Water	112
Normal	98
Grass	70
Bug	69
Psychic	57
Fire	52
Rock	44
Electric	44
Ground	32
Dragon	32
Ghost	32
Dark	31
Poison	28
Steel	27
Fighting	27
Ice	24
Fairy	17
Flying	4

```
# Show the proportion of unique pokemon types
pokemon['Type 1'].value_counts(normalize=True)
```

Water	0.14000
Normal	0.12250
Grass	0.08750
Bug	0.08625
Psychic	0.07125
Fire	0.06500
Rock	0.05500
Electric	0.05500
Ground	0.04000
Dragon	0.04000
Ghost	0.04000
Dark	0.03875
Poison	0.03500
Steel	0.03375
Fighting	0.03375
Ice	0.03000
Fairy	0.02125
Flying	0.00500

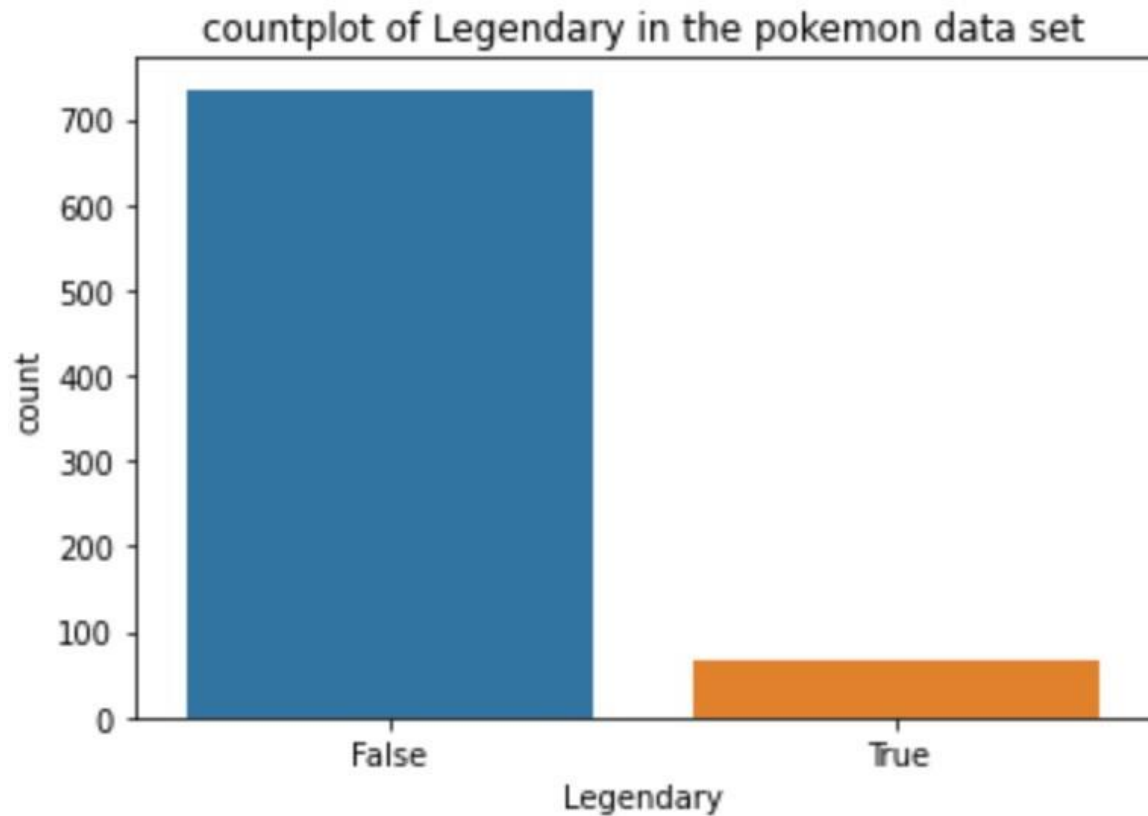
Exploring Categorical Variables



Exploring Categorical Variables

- Measuring the **central tendency** – the mode
 - The **mode** of a variable is the value occurring most often.
 - It would be dangerous to place too much emphasis on the mode, since the most common value is not necessarily a majority.

Exploring Categorical Variables



Overview

- Exploring the Structure of Data
- Exploring Numeric Variables
- Exploring Categorical Variables
- Exploring Relationships between Variables

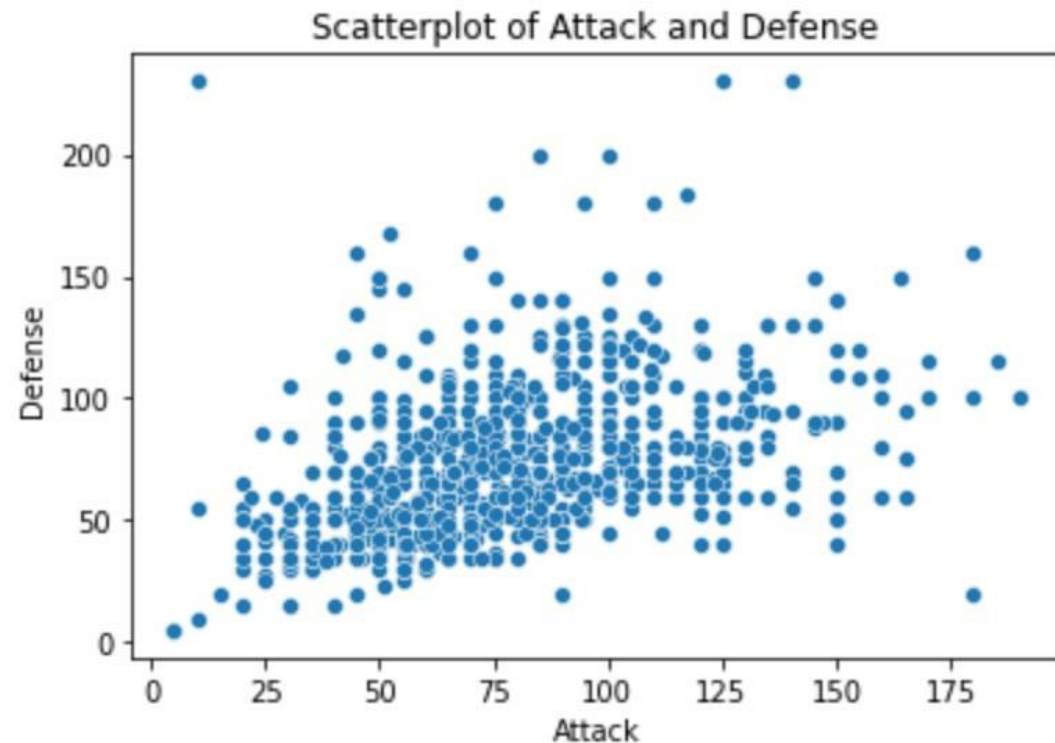
Exploring relationships between variables

- So far, we have examined variables one at a time, calculating only **univariate** statistics.
 - **Bivariate** relationships consider the relationship between two variables. Relationships of more than two variables are called **multivariate** relationships.

Exploring relationships between variables

■ Scatterplots: two numeric variables

- A **scatterplot** is a diagram that visualizes a bivariate relationship.
 - 2-dimensional feature space



Exploring relationships between variables

■ Scatterplots: two numeric variables

- Positive/Negative association.
- The strength of a linear association between two variables is measured by a statistic known as **correlation** (range [-1, 1]).

```
pokemon[['Attack', 'Defense']].corr()
```

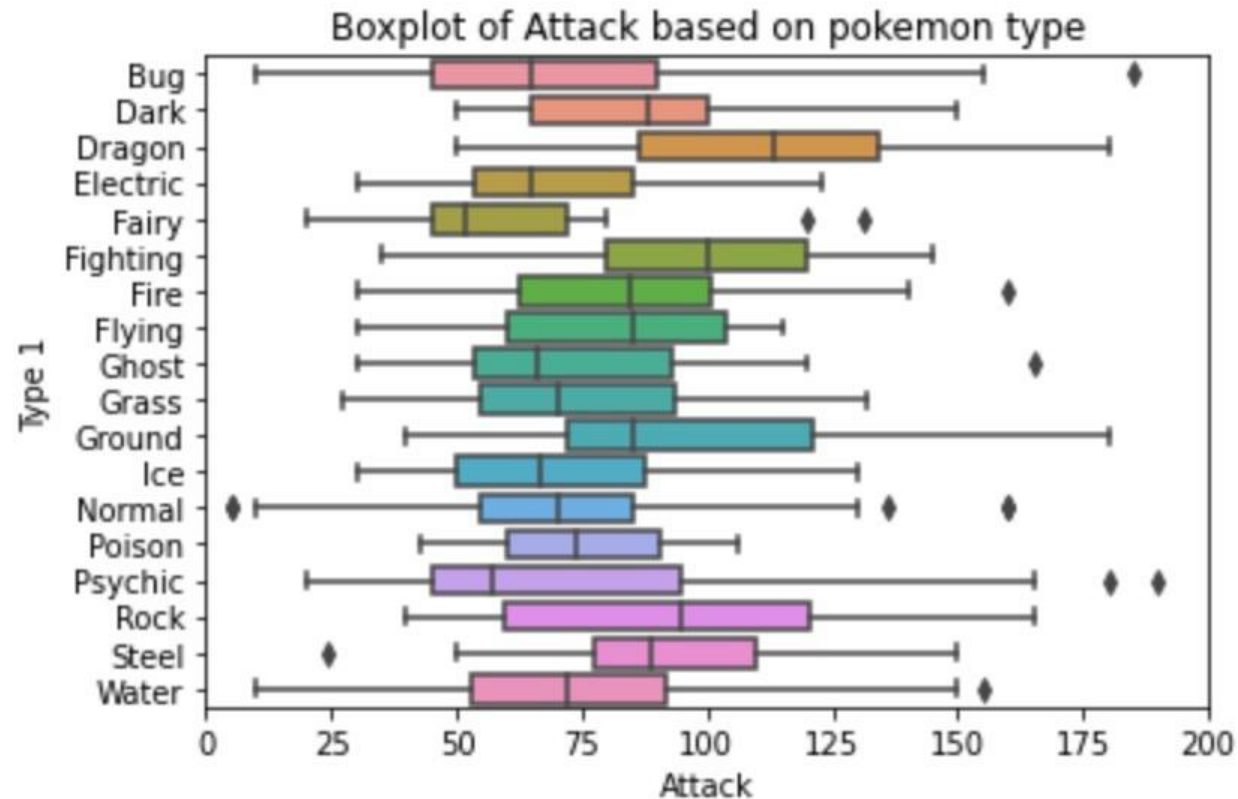
	Attack	Defense
Attack	1.000000	0.438687
Defense	0.438687	1.000000

```
pokemon.corr()
```

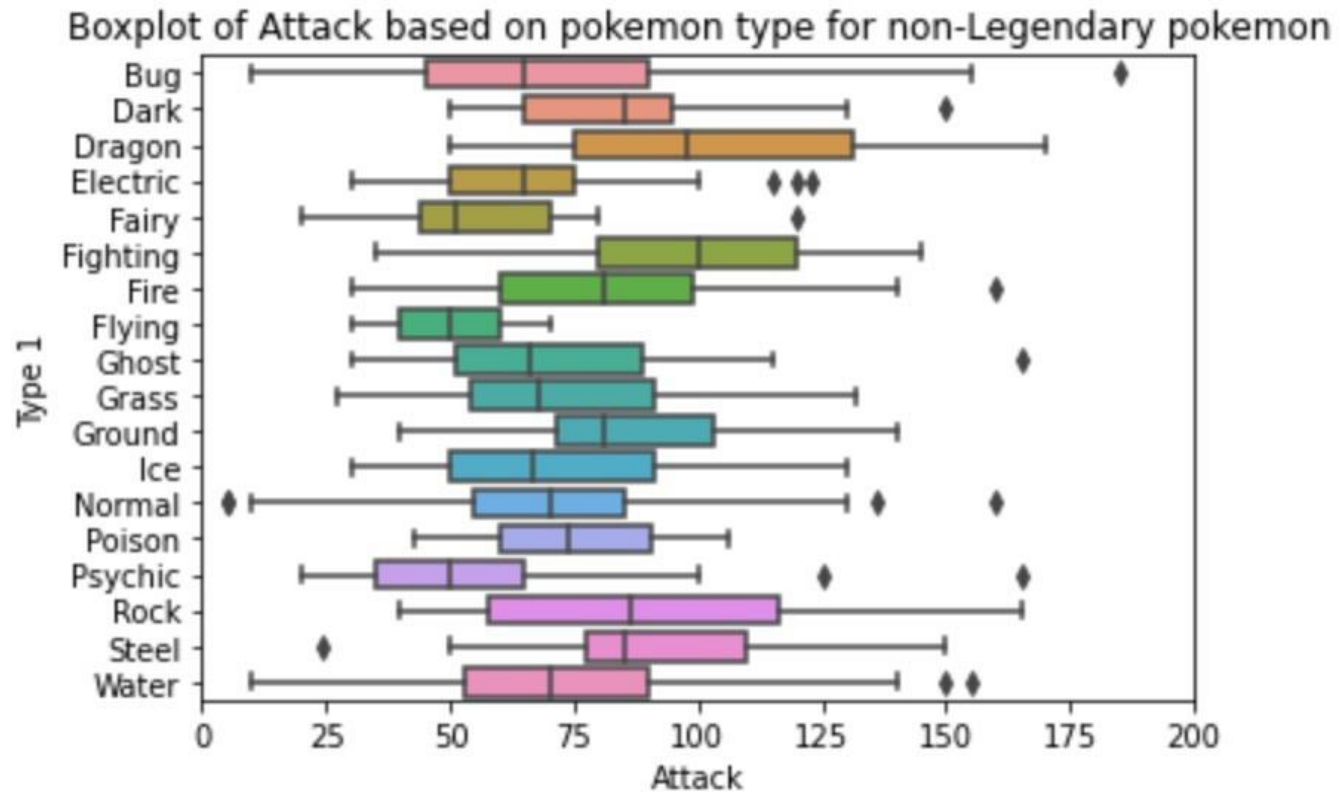
	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
HP	1.000000	0.422386	0.239622	0.362380	0.378718	0.175952
Attack	0.422386	1.000000	0.438687	0.396362	0.263990	0.381240
Defense	0.239622	0.438687	1.000000	0.223549	0.510747	0.015227
Sp. Atk	0.362380	0.396362	0.223549	1.000000	0.506121	0.473018
Sp. Def	0.378718	0.263990	0.510747	0.506121	1.000000	0.259133
Speed	0.175952	0.381240	0.015227	0.473018	0.259133	1.000000

Exploring relationships between variables

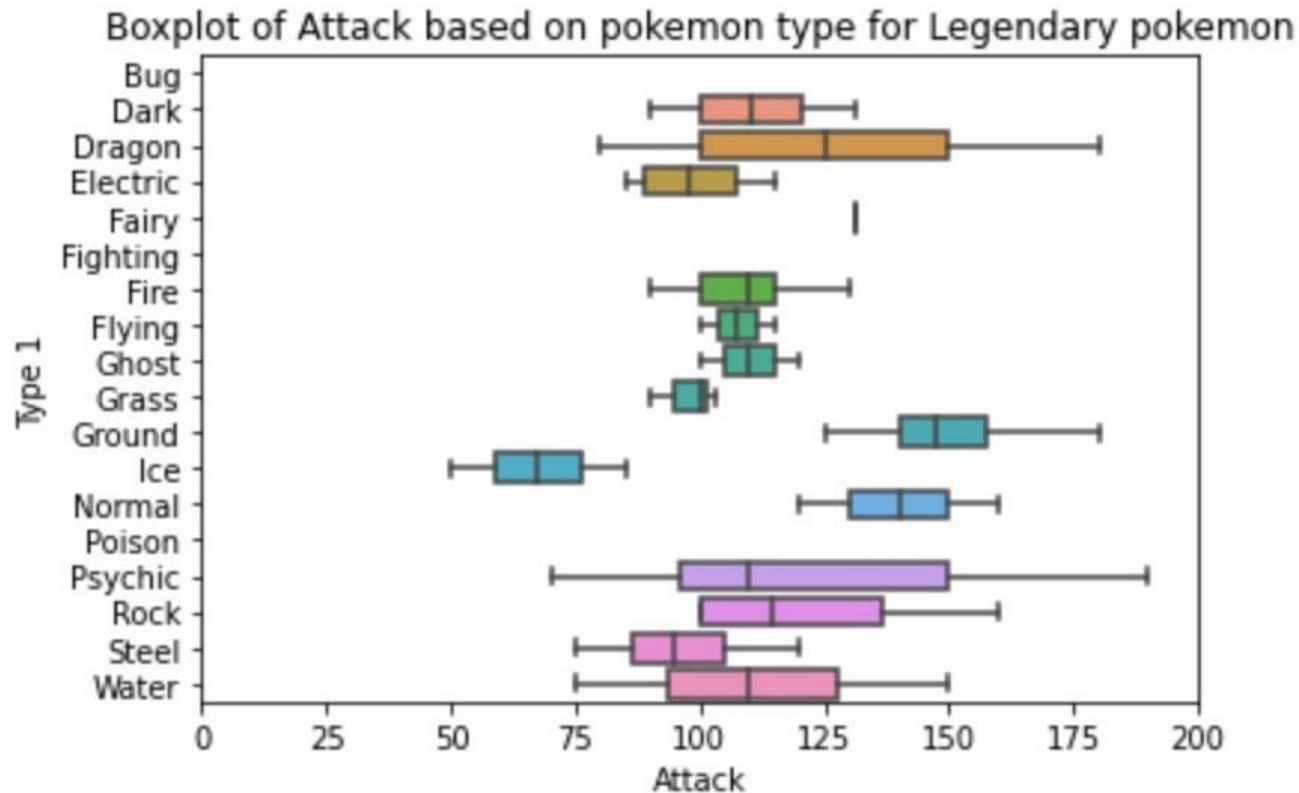
■ Relationships between numeric variable and categorical variable



Exploring relationships between variables



Exploring relationships between variables



What does this chart have to say about a legendary Pokémon's attack and type?